Getting to grips with Airflow on Amazon AWS

Cite as: Martin Paul Eve, "Getting to grips with Airflow on Amazon AWS", *eve.gd: Martin Paul Eve*, February 13, 2023, https://doi.org/10.59348/c41fh-cp382.

I am currently conducting a research project at Crossref that requires me to build a database using large backend files (e.g. building a relational database from a 3GB XML file). We need to rebuild this monthly, so Apache Airflow seemed a good tool to run these periodic tasks.

There are, however, lots of "gotchas" in this framework that can trip up a newcomer and I thought it might be helpful to document some of these.

You can't pass data between tasks

Airflow uses a system called XCOM to transfer data between tasks, but it's only suitable for very small quantities. So you can't get the output of one task and easily feed it to another. Instead, you have to pass *locations* of data. (To my mind, this makes the chunking into micro-tasks much harder.)

You can't create a dynamic number of parallel tasks

I wanted to do something like this:

```
for output in task_one():
        task_two()
```

with all "task_two"s running in parallel. This won't work. Don't try it.

If you want to import other packages, you need to package them

To get imports working, you have to pip-up your other package.

If you have a settings file that externalises secrets or similar, you can get that into the system like this:

```
import os
import sys
import inspect
from pathlib import Path

current_directory =
os.path.dirname(os.path.abspath(inspect.getfile(inspect.currentframe())))

settings_file = Path(current_directory) / 'downloaded_settings.py'

logging.info('Settings file downloading to: '
             '{}'.format(settings_file))

import boto3
s3client = boto3.client('s3')
s3client.download_file(CODE_BUCKET, 'settings.py',
                       str(settings_file))

sys.path.append(current_directory)
os.environ.setdefault('DJANGO_SETTINGS_MODULE',
                      'downloaded_settings')
```

You can't use the @task.virtualenv decorator "out of the box"

You need to create a package.zip that contains a virtual_python_plugin.py. You also need to update this script so that it points to the correct python version for your install. See the Amazon sample for this.

The "small" instance only has 2GB of RAM and you can run out of memory quickly

So processing even moderately large files has to be done using generators/streaming approaches. Using an ORM's atomic transaction mode (say, @atomic.transaction, in Django) creates a set of in-memory changes. This can also quickly result in out-of-memory errors.

The OOM killer/errors present themselves as "SIGKILL: 9" messages in the log.

You can also get OOM errors if your application usage spikes and is combined with the memory usage of the Airflow web server. For instance, on the "Small" instance, the web server was using 35% of available memory. When the BaseWorker hit above 60% of memory usage, the combined total available memory caused the task to fail.

You can monitor memory usage at CloudWatch by adding metrics from Cluster -> MWAA.

You can also try explicitly invoking the garbage collector at various points (gc.collect()). *This worked for me*.

If parsing XML, use ElementTree.iterparse *but* also call .clear() on *every* element at the end of the parsing event loop.

If using the Django ORM beware of the query log filling up memory (call django.db.reset_queries() periodically).

Your execution role needs permissions to access secrets

Something like this (probably best restricted to specific secrets):

```
{
    "Effect": "Allow",
    "Action": [
        "secretsmanager:GetSecretValue*"
    ],
    "Resource": [
        "*"
    ]
}
```

Excessive logging can lead the worker to crash

When I had a really huge amount of logging enabled, I got an Unexpected SSL error on the Airflow Postgres backend. This crashed my task.

Passing certain variables to your tasks causes a Jinja templating crash

This code crashes:

```
CODE_BUCKET = 'airflow-crossref-research-annotation'
PORTICO_URL = 'https://api.portico.org/kbart/Portico_Holding_KBart.txt'

@task.virtualenv(task_id="import_portico",
                     requirements=["requests", "boto3",
                                   f"preservation-database=={DATABASE_VERSION}",
                                   "psycopg2-binary",
                                   "crossrefapi", "django"],
                     system_site_packages=True)
    def import_portico(code_bucket):
        pass

t1 = import_portico(CODE_BUCKET, PORTICO_URL)
```

This code does not:

```
CODE_BUCKET = 'airflow-crossref-research-annotation'

@task.virtualenv(task_id="import_portico",
                     requirements=["requests", "boto3",
                                   f"preservation-database=={DATABASE_VERSION}",
                                   "psycopg2-binary",
                                   "crossrefapi", "django"],
                     system_site_packages=True)
    def import_portico(code_bucket):
        PORTICO_URL = 'https://api.portico.org/kbart/Portico_Holding_KBart.txt'
        pass

t1 = import_portico(CODE_BUCKET)
```

Go figure.